Monte Carlo Control and SARSA on TextFlappyBird: A Comparative Analysis

Jean-Vincent Martini

CentraleSupélec

Abstract. This report evaluates Monte Carlo Control and SARSA algorithms on state- and screen-based versions of the TextFlappyBird environment. We analyze hyperparameter sensitivity, investigate policy transferability across configurations, and compare agent performance using rewards and scores. The report concludes with a discussion of observed trends and potential improvements. The code is available at https://github.com/JeanJNMV/MDS-RL_Text_Flappy_Bird.

Keywords: Reinforcement Learning · Deep Learning · Monte Carlo Control · SARSA · TextFlappyBird

1 Introduction and Problem Definition

The objective of this project is to apply reinforcement learning (RL) techniques to master a simplified, terminal-based variant of the popular game Flappy Bird, known as Text Flappy Bird (TFB). In this environment, the player controls a simple unit-element character navigating through a series of pipe obstacles, aiming to maximize survival time and score.

The problem is modeled using two distinct variations of the TFB environment, provided via a custom OpenAI Gym implementation: the simple version and the screen version. Each version presents unique challenges in terms of observation space and complexity, requiring different approaches to policy learning and value estimation. To solve this environment, we formulate the gameplay as a Markov Decision Process and investigate two specific RL algorithms: the Monte Carlo Control method and the Sarsa(λ) algorithm.

By implementing these agents, this report aims to evaluate and compare their convergence and overall performance across both environment versions. Furthermore, we conduct a sensitivity analysis on key hyperparameters and investigate the transferability of the learned policies to provide deeper insights into the behavior of both algorithms.

2 Implementation Details

Environment Setup and State Representation. Both TFB variants share the same dynamics: the action space consists of two actions, 0 (Idle) and 1 (Flap), the agent receives a reward of +1 per step alive (making the reward proportional to the score since the score increases by 1 for every 10 steps) and an episode

terminates on collision. They differ only in their observation space. The simple version provides a tuple (dx, dy) representing the horizontal and vertical distances to the center of the nearest pipe gap, restricting the state space to the most critical navigational information. The screen version instead exposes the full 20×15 game grid as a `gym.spaces.Box`, where each cell encodes an entity: 0 (background), 1 (alive player), 2 (pipe), or 3 (dead player), requiring the agent to implicitly extract geometric features from the raw spatial layout.

Agents and Algorithms. We implemented two classical RL algorithms. Both utilize an ϵ-greedy exploration strategy with exponential decay. For the screen environment, two complementary state representations are used. In the hash-based variant, the raw grid array is serialized into a nested tuple usable as a dictionary key, preserving full grid information at the cost of a large, non-generalizing state space. In the tile-coding variant, a compact feature vector (d_x, d_y) is extracted from the screen and encoded using 8 overlapping tilings over a 4×4 grid, enabling generalization across nearby states through linear function approximation.

The Monte Carlo Control (MCC) agent learns from complete episodes. At the end of each episode, the cumulative discounted return G_t is computed backwards from the terminal state. The Q-values are updated for every visited state-action pair using a constant step-size α: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(G_t - Q(S_t, A_t))$.

The Sarsa(λ) agent, in contrast, utilizes eligibility traces to propagate delayed rewards back to recently visited states more efficiently[5]. Our implementation increments an accumulating trace $E(s, a)$ upon visitation. The error δ_t and subsequent Q-value updates for all active traces are defined as: $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$ and $Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t E(s, a)$. To optimize memory and computational performance during training, any traces that decay below a negligible threshold (1×10^{-10}) are dynamically deleted.

3 Evaluation and Results

Performance Comparison between Monte Carlo Control and SARSA. Agents were trained for 25 000 episodes and evaluated over 200 episodes (fewer for tile coding due to slower inference) on the training setup (`height=15`, `width=20`, `pipe_gap=4`), with a 50 000-step cap per episode (max score: 4999). Figure 1 shows training curves and Table 1 reports results. Due to a heavily skewed score distribution, the median better reflects typical performance than the mean, which is influenced by rare high scores and often too close to the standard deviation.

In the simple environment, both agents learned effective policies, but MCC was more stable at evaluation, consistently reaching 4999, while Sarsa remained less stable despite similar training scores. In the screen environment, tile coding proved to be essential. Both MCC and Sarsa with tile coding achieved maximal evaluation performance, whereas hash-based agents failed to generalize across the large state space, producing much lower scores. The gap in performance between MCC and Sarsa could stem from Sarsa's bootstrapping, where Q-updates

depend on current next-state estimates, introducing variance, while MCC uses full episode returns, yielding lower-variance and more reliable estimates in this case.

Figure 2 compares learned value functions and policies. In the simple and tile-coded settings, both agents learned similar value landscapes, with high values near the screen center (pipe gap), and converged almost to the same policy: often flap below the gap, idle above. With hash-based representation, no meaningful value function or policy could be visualized due to the large sparse state space. We can still observe that most states had zero value estimates, indicating failure to learn a coherent strategy.

Hyperparameter Sensitivity Analysis. We conducted a sensitivity analysis on key hyperparameters, including the learning rate α, discount factor γ, and exploration rate ϵ. We only considered the simple environment and the tile-based screen environment, as the hash-based agents did not learn meaningful policies. For each agent and environment, we varied one hyperparameter at a time while keeping the others fixed at their original values. The number of training episodes was reduced to 5 000 and the number of explored values per hyperparameter was limited due to computational constraints. Figure 4 in the Appendix shows the results of this analysis. TODO: add details on the specific values explored for each hyperparameter and the results of the sensitivity analysis.

Transferability of Learned Policies through Different Environment Configurations. Here we evaluate the generalization of the learned policies across different environment configurations, specifically varying the screen dimensions and pipe gap. We trained the 3 variants of our best performing agent, the MCC, on the original configuration (`height=15`, `width=20`, `pipe_gap=4`) and evaluated them on 7 new configurations, including shorter and taller screens and pipe gaps. Figure 3 shows the results of this evaluation. Both the simple and tile-based agents showed the same degree of generalization, with strong performance for a wider pipe gap and a slightly shorter screen, but a significant drop in performance for all other configurations. This suggests that the learned policies rely heavily on the specific geometry of the training environment, and that even small changes in the layout can significantly impact performance. The hash-based agent, which already struggled to learn a good policy in the original configuration, performed poorly across all new configurations, further highlighting its lack of generalization capabilities.

Key Limitations for the Real Flappy Bird Game. Although some agents perform well in TFB, transferring them to the original Flappy Bird Gym faces key limitations. Agents in the simple environment depend on a compact (dx, dy) observation not provided by the original game, requiring feature extraction. Hash-based screen agents are highly unfit for this task, as changes in resolution or layout create unseen states and invalidate the Q-table. Tile-coding agents are more promising due to their use of (dx, dy) features and robustness to small

variations, but they still require careful parameter tuning and may struggle with increased complexity, noise, faster dynamics, and richer visuals. More generally, all methods rely on tabular or linear representations that do not scale well to the original game's visual complexity. Prior work on Flappy Bird includes DQN-based methods [1,3], genetic algorithms [4], tabular Q-learning [6], and neuroevolution [2].

4 Conclusion

This study shows that both Monte Carlo Control and Sarsa can learn effective policies in TextFlappyBird, with MCC demonstrating greater stability and reliability overall. Feature representation plays a critical role, as tile coding enables strong generalization while hash-based methods fail in high-dimensional settings. Finally, limited transferability across environment variations highlights the need for more robust function approximation methods to handle increased complexity.

A Figures

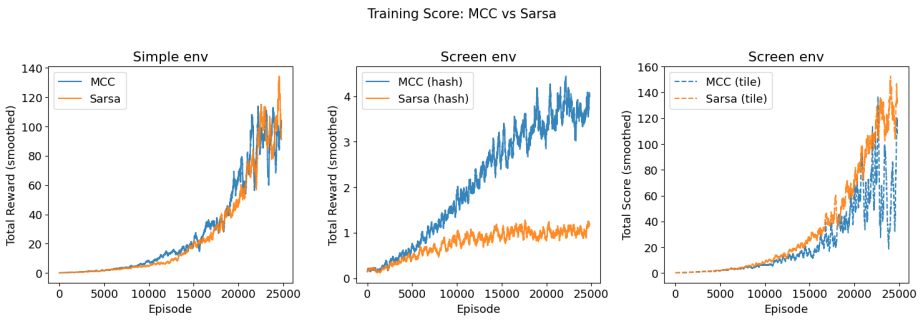


Fig. 1: Training scores for all agents and environment configurations. The curves are smoothed using a moving average with a window size of 100 episodes for better visualization.

Agent	Env	Mean Rew.	Std Rew.	Mean Score	Std Score	Med. Rew.	Med. Score
MCC	v0	50000.0	0.0	4999.00	0.00	50000.0	4999.00
Sarsa	v0	7489.6	8028.2	747.66	802.82	4338.0	432.50
MCC	screen-v0	53.5	36.8	4.11	3.67	43.0	3.00
Sarsa	screen-v0	22.9	14.5	1.06	1.47	19.0	1.00
MCC Tile	screen-v0	50000.0	0.0	4999.00	0.00	50000.0	4999.00
Sarsa Tile	screen-v0	50000.0	0.0	4999.00	0.00	50000.0	4999.00

Table 1: Evaluation results per agent and environment configuration. The maximum possible reward is 50 000 (corresponding to a score of 4999) due to the step cap.

B Figures for Hyperparameter Sensitivity Analysis

References

1. Appiah, N., Vare, S.: Playing flappybird with deep reinforcement learning. Playing FlappyBird with Deep Reinforcement Learning. Junthbasnet (2018)
2. Brandão, A., Pires, P., Georgieva, P.: Reinforcement learning and neuroevolution in flappy bird game. In: Iberian Conference on Pattern Recognition and Image Analysis. pp. 225–236. Springer (2019)
3. Chen, K.: Deep reinforcement learning for flappy bird. CS 229 Machine-Learning Final Projects (2015)

4. Gan, R.: Flappy bird: Optimization of deep q-network by genetic algorithm. In: 2022 IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA). pp. 703–707. IEEE (2022)
5. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. The MIT Press, Cambridge, Massachusetts, second edn. (2018)
6. Vu, T., Tran, L.: Flapai bird: training an agent to play flappy bird using reinforcement learning techniques. arXiv preprint arXiv:2003.09579 (2020)

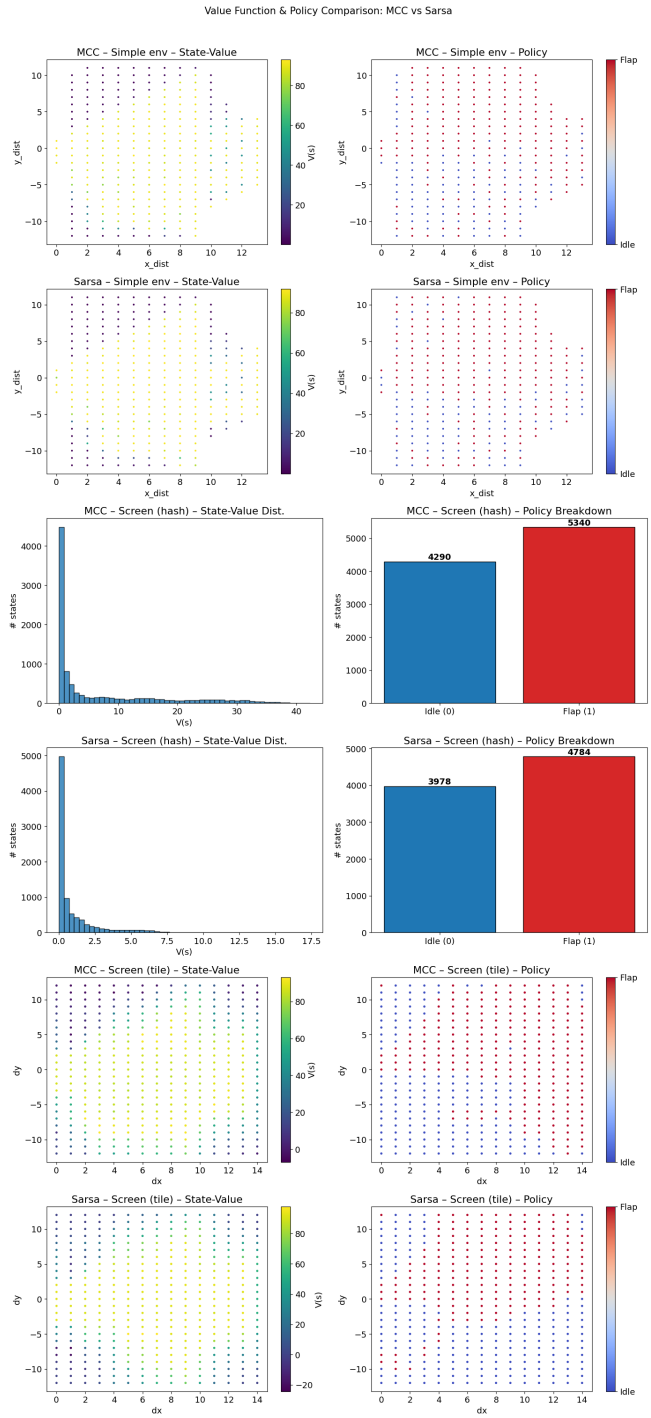


Fig. 2: State-value function estimates (left) and Policy visualization (right) for all agents. For the screen environment with the hash-based representation, since the number of states is extremely large, we only visualize the repartition of value estimates across the state in the left column and the repartition between the two actions in the right column.

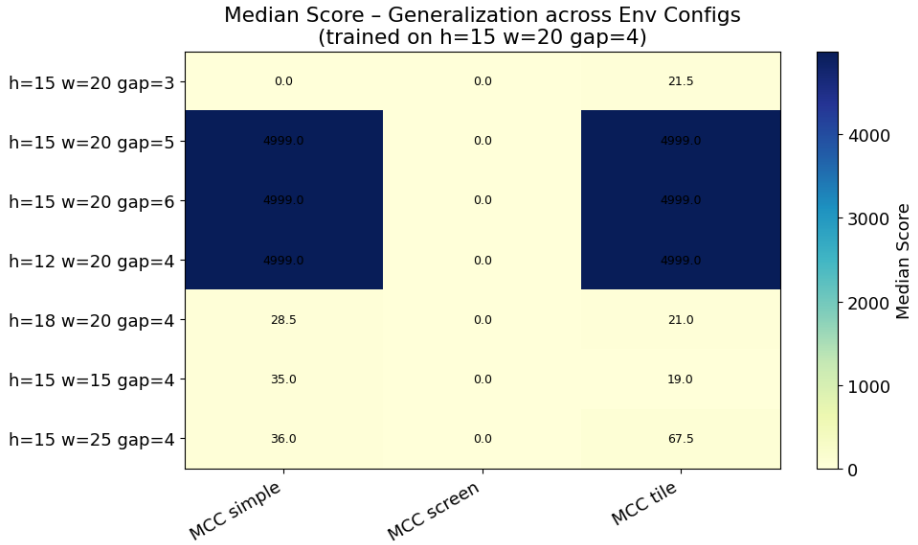
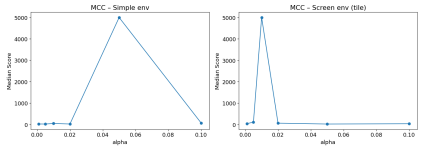
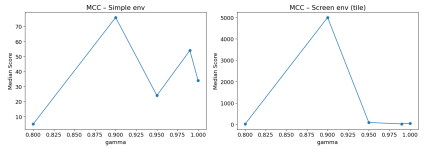


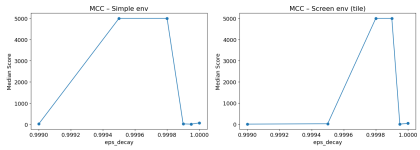
Fig. 3: Generalization performance of the MCC agents across different environment configurations. Each section of the plot corresponds to a different configuration on which the agents were evaluated after being trained on the original configuration (`height=15`, `width=20`, `pipe_gap=4`).



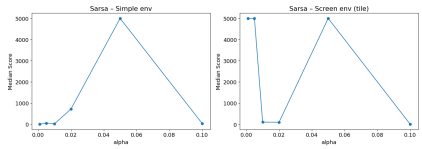
(a) MCC - Learning Rate α



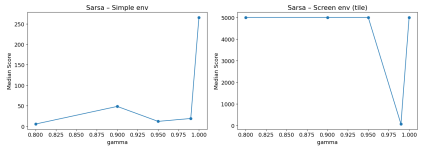
(b) MCC - Discount Factor γ



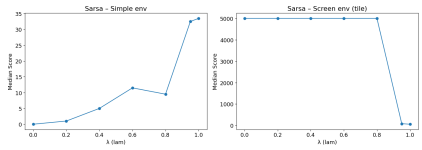
(c) MCC - Exploration Decay ϵ



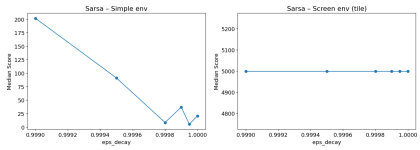
(d) Sarsa - Learning Rate α



(e) Sarsa - Discount Factor γ



(f) Sarsa - Trace Decay λ



(g) Sarsa - Exploration Decay ϵ

Fig. 4: Sensitivity analysis of key hyperparameters for the MCC and Sarsa agents. Each plot shows the evaluation performance (score) as a function of the hyperparameter being varied, while keeping the other hyperparameters fixed at their original values. The base configuration used is: $\alpha = 0.01$, $\gamma = 0.99$, $\lambda = 0.8$ (for Sarsa), and ϵ decay of 0.9998.